

PÓS-GRADUAÇÃO FIAP — POSTECH

ARQUITETURA E DESENVOLVIMENTO DE SISTEMAS DE SOFTWARE

FIAP RESTAURANTE — FASE 3

Documentação de arquitetura do sistema de pedidos on-line para restaurante

Danilo Fernando de Paula e Silva

Gilmar da Costa Moraes Junior

Juliana Maria Dal Olio Braz

Luis Henrique Silveira Borges

Thiago de Jesus Cordeiro

São Paulo — Maio de 2026

Danilo Fernando de Paula e Silva
Gilmar da Costa Moraes Junior
Juliana Maria Dal Olio Braz
Luis Henrique Silveira Borges
Thiago de Jesus Cordeiro

FIAP RESTAURANTE — FASE 3

Documentação de arquitetura do sistema de pedidos on-line para restaurante

Documentação técnica apresentada ao programa de Pós-Graduação FIAP — PosTech como parte das entregas do Tech Challenge da Fase 3. O sistema descrito implementa um conjunto de microsserviços para gerenciamento de pedidos, integração com gateway externo de pagamento e fluxo de produção da cozinha.

São Paulo — Maio de 2026

ACESSO AO CÓDIGO E AO VÍDEO

Repositório no GitHub

<https://github.com/lh-borges/fiap-restaurante-fase3>

Código-fonte completo, histórico de PRs, coleção Postman, ADRs e diagramas. Clone o repositório e execute `docker compose up -d --build` na raiz para subir o sistema inteiro com um único comando.

Vídeo de apresentação no YouTube

Aguardando publicação após a gravação.

O vídeo de apresentação técnica (até 10 minutos) está em fase de gravação. O roteiro detalhado, com falas, tempos e comandos exatos, está disponível em `docs/roteiro-video.md` e cobre os 7 blocos da apresentação. A URL desta seção será atualizada após a publicação no YouTube.

RESUMO

Este documento descreve a arquitetura de um sistema distribuído desenvolvido como entrega do Tech Challenge da Fase 3 do programa de Pós-Graduação FIAP — PosTech. O sistema implementa o fluxo on-line de pedidos para um restaurante, incluindo cadastro e autenticação de clientes, criação e confirmação de pedidos, comunicação com um gateway externo eventualmente disponível para processamento de pagamentos, tratamento de falhas com resiliência e propagação do pedido para a cozinha após a aprovação do pagamento. A solução é composta de quatro microsserviços Spring Boot 4 / Java 25, comunicando-se de forma síncrona via gRPC e GraphQL e assíncrona via Apache Kafka, com persistência em MySQL 8.4 isolada por bounded context. A resiliência da chamada ao gateway é provida pela biblioteca Resilience4j (Circuit Breaker, Retry, Timeout e Fallback), e a recuperação automática é garantida por um worker agendado. Toda a orquestração local é provida por Docker Compose, permitindo execução do sistema completo com um único comando.

Palavras-chave: microsserviços; arquitetura hexagonal; Apache Kafka; Resilience4j; Spring Boot; GraphQL; Docker; JWT.

SUMÁRIO

ACESSO AO CÓDIGO E AO VÍDEO	3
RESUMO	4
SUMÁRIO	5
1 INTRODUÇÃO	8
2 VISÃO GERAL DO SISTEMA	8
2.1 Domínio do problema	8
2.2 Requisitos atendidos	9
2.3 Componentes externos	9
3 ARQUITETURA	9
3.1 Visão de componentes	9
3.2 Microserviços	11
3.2.1 <i>usuario-autenticacao</i>	12
3.2.2 <i>restaurante-pedido</i>	12
3.2.3 <i>pagamento-service</i>	12
3.2.4 <i>restaurante-service</i>	12
3.3 Infraestrutura	12
3.3.1 <i>Apache Kafka em modo KRaft</i>	12
3.3.2 <i>MySQL</i>	13
3.3.3 <i>procpag</i>	13
4 FLUXO PRINCIPAL	13
4.1 Máquina de estados do agregado Pedido	15
5 GUIA DE INTEGRAÇÃO	16
5.1 Visão geral do contrato	16
5.2 Endpoints disponíveis	17
5.3 Fluxo recomendado de integração	17
5.4 Classificação dos erros	18
5.5 Como descobrir o schema em runtime	18
6 RESILIÊNCIA E TOLERÂNCIA A FALHAS	18
6.1 Padrões aplicados	19

6.2 Worker de reprocessamento	20
6.3 Garantias e limitações	20
7 SEGURANÇA	20
7.1 Autenticação e autorização	20
7.2 Ownership check em pedidos	21
8 DECISÕES ARQUITETURAIS	21
9 ESTRATÉGIA DE TESTES	22
10 OPERAÇÃO E DEPLOY	22
10.1 Limites de recursos	22
10.2 Observabilidade	23
10.3 Performance	23
11 CONSIDERAÇÕES FINAIS	23
REFERÊNCIAS	24

1 INTRODUÇÃO

Os sistemas de pedidos on-line para restaurantes evoluíram rapidamente nos últimos anos, pressionados pelo crescimento do delivery e pela necessidade de integração com gateways externos de pagamento, plataformas de logística e displays operacionais nas cozinhas. Esses sistemas precisam ser **resilientes** (toleram a indisponibilidade temporária de serviços de terceiros), **escaláveis** (suportam picos como horário de almoço sem degradar a experiência) e **seguros** (proteção dos dados do cliente e dos pagamentos).

Este documento apresenta a arquitetura do **FIAP Restaurante**, um sistema distribuído desenvolvido como Tech Challenge da Fase 3 do programa de Pós-Graduação FIAP — PosTech. O sistema cobre o ciclo completo de um pedido: cadastro do cliente, autenticação, criação e confirmação do pedido, comunicação com um gateway externo eventualmente disponível, tratamento de falhas, e propagação do pedido para a cozinha até sua finalização.

A solução é composta de quatro microsserviços Spring Boot 4 / Java 25, conectados por um broker Apache Kafka em modo KRaft e persistindo em databases isolados sobre uma instância MySQL 8.4. Toda a infraestrutura é descrita em um único arquivo **docker-compose.yml**, que sobe o sistema completo (oito containers) com um único comando.

Este documento detalha a arquitetura, as decisões técnicas que a guiaram, os mecanismos de resiliência implementados, a estratégia de testes adotada e os procedimentos de operação. Decisões pontuais estão também registradas em ADRs (Architecture Decision Records) na pasta `docs/adr/` do repositório.

2 VISÃO GERAL DO SISTEMA

2.1 Domínio do problema

O sistema modela quatro **bounded contexts** da operação de um restaurante on-line:

- **Identidade do cliente:** cadastro, autenticação e emissão de tokens JWT.
- **Pedido:** criação, cálculo de valor total, confirmação, consulta e atualização de status conforme eventos externos.
- **Pagamento:** processamento da cobrança contra um gateway externo eventualmente disponível, com tratamento de falhas e reprocessamento automático.
- **Cozinha (produção):** fila de pedidos aprovados, com transições de estado disparadas pelo dono do restaurante (recebido, em preparo, pronto).

Cada um desses contextos foi materializado em um microsserviço independente, com modelo de dados próprio e fronteiras explícitas. A comunicação entre eles privilegia eventos assíncronos via Apache Kafka, recorrendo a chamadas síncronas (gRPC) apenas quando a operação exige.

2.2 Requisitos atendidos

Os requisitos formais da Fase 3 estão sumarizados a seguir, com indicação do componente responsável por cada um:

Req.	Descrição	Componente
4.1	Cadastro e autenticação de clientes	usuario-autenticacao
4.2	Criação e confirmação de pedido	restaurante-pedido
4.3	Consulta de pedido por ID e listagem por cliente	restaurante-pedido
4.4	Processamento de pagamento via gateway externo	pagamento-service + procpag
4.5	Marcação de pagamento pendente em falha	pagamento-service (Resilience4j)
4.6	Reprocessamento automático ao restabelecimento	pagamento-service (@Scheduled)
4.7	Atualização automática do status do pedido	restaurante-pedido (consumer)
5.1	Arquitetura em múltiplos serviços	4 microserviços
5.2	Spring Security + JWT (RS256)	todos os serviços de aplicação
5.3	Comunicação assíncrona via Kafka	6 tópicos
5.4	Resiliência (CB + Retry + Timeout + Fallback)	pagamento-service.ExternalPaymentClient
5.5	Clean / Hexagonal Architecture	todos os 4 módulos

Além do mínimo exigido, foi implementado o módulo opcional **restaurante-service** (item 5.1 da spec), responsável pelo bounded context da cozinha.

2.3 Componentes externos

O sistema integra-se a um gateway externo de pagamento denominado **procpag**, fornecido pelos professores como uma imagem Docker (docker.io/erickemprobr/procpag). Esse serviço é deliberadamente instável: ora aprova, ora responde com erro ou timeout, simulando um gateway real eventualmente disponível. Toda a estratégia de resiliência da solução é desenhada em torno dessa imprevisibilidade.

3 ARQUITETURA

3.1 Visão de componentes

A arquitetura é composta de oito containers em rede privada Docker. Quatro deles são as aplicações Spring Boot (uma por bounded context), três são peças de infraestrutura (MySQL, Kafka e Kafka UI) e o oitavo é o gateway externo procpag.

A arquitetura está documentada em três níveis complementares, seguindo o **modelo C4** de Simon Brown:

- **C4 nível 1 — Contexto** (Figura 1): o sistema visto de fora; atores e sistemas externos.
- **C4 nível 2 — Containers** (Figura 2): a topologia técnica interna; aplicações Spring, MySQL, Kafka, com protocolos e tecnologias.
- **Diagrama de componentes** (Figura 3): visão de componentes lógicos com o detalhamento dos eventos Kafka.

Diagrama de Contexto — FIAP Restaurante (Fase 3)

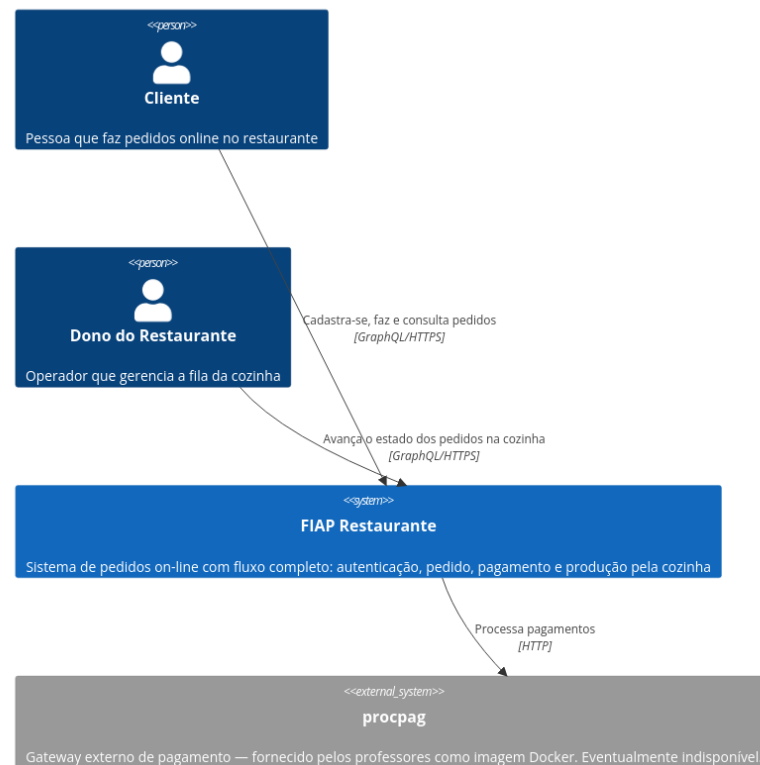


Figura 1 — Diagrama de Contexto (C4 nível 1).

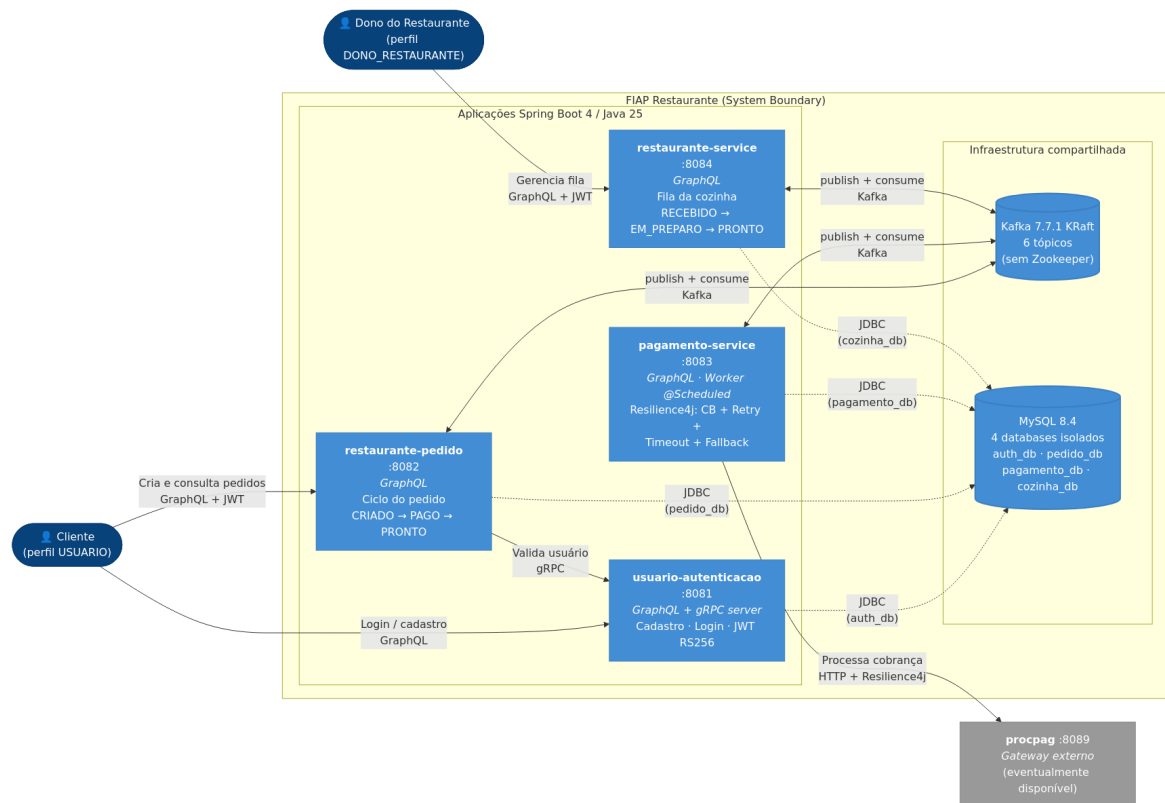


Figura 2 — Diagrama de Containers (C4 nível 2).

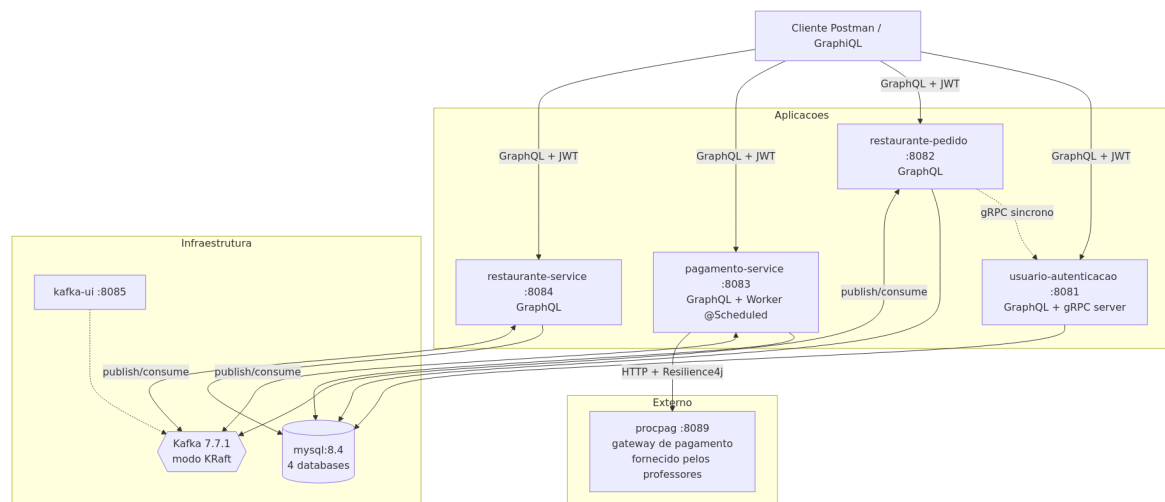


Figura 3 — Diagrama de componentes (visão lógica complementar).

3.2 Microserviços

Cada microserviço expõe sua API pública via GraphQL (Spring GraphQL), com GraphiQL embutido na URL `/graphiql`. Schemas residem em `src/main/resources/graphql/schema.graphqls` de cada módulo.

3.2.1 usuario-autenticacao

Responsável por cadastro de usuários, autenticação, emissão de JWT (RS256) e consulta interna via gRPC. Persiste no database `auth_db`. Não consome nem publica eventos Kafka. Expõe servidor gRPC na porta interna 9000 para os demais serviços consultarem dados de usuário de forma síncrona.

3.2.2 restaurante-pedido

Responsável pelo ciclo de vida do pedido: criação, cálculo de valor total, confirmação, consulta por ID, listagem por cliente autenticado e atualização automática de status conforme eventos vindos do pagamento e da cozinha. Persiste no database `pedido_db`. É o componente que publica os eventos `pedido.criado` (disparando o pagamento) e `pedido.pronto-para-cozinha` (após confirmação do pagamento).

3.2.3 pagamento-service

Consome o evento `pedido.criado` e tenta processar a cobrança contra o gateway externo procpag. Aplica Resilience4j (Circuit Breaker, Retry, Timeout, Fallback). Em caso de falha, marca o pagamento como pendente e publica `pagamento.pendente`; em caso de sucesso, publica `pagamento.aprovado`. Hospeda também o worker `@Scheduled` que, a cada 30 segundos, drena os pagamentos pendentes tentando novamente a chamada ao gateway. Persiste no database `pagamento_db`.

3.2.4 restaurante-service

Modela o bounded context da cozinha. Consome `pedido.pronto-para-cozinha` e cria um agregado `PedidoCozinha` no estado RECEBIDO. O dono do restaurante (perfil `DONO_RESTAURANTE` no JWT) avança o estado via mutations GraphQL: `iniciarPreparo` (RECEBIDO → EM_PREPARO) e `marcarComoPronto` (EM_PREPARO → PRONTO). Cada transição publica um evento (`pedido.em-preparo` ou `pedido.pronto`) consumido pelo `restaurante-pedido` para refletir o status no agregado principal. Persiste no database `cozinha_db`.

3.3 Infraestrutura

3.3.1 Apache Kafka em modo KRaft

O broker roda em **modo KRaft** (Kafka Raft, GA desde Kafka 3.3), eliminando a dependência de Zookeeper. Em configuração **single-node** apropriada para desenvolvimento e demonstração, com replication factor 1 nos tópicos internos. O ADR 0006 detalha a decisão.

São **seis tópicos** orquestrando o fluxo:

Tópico	Publicador	Consumidor(es)
pedido.criado	restaurante-pedido	pagamento-service
pagamento.aprovado	pagamento-service	restaurante-pedido
pagamento.pendente	pagamento-service	restaurante-pedido
pedido.pronto-para-cozinha	restaurante-pedido	restaurante-service
pedido.em-preparo	restaurante-service	restaurante-pedido
pedido.pronto	restaurante-service	restaurante-pedido

A chave de cada mensagem é o `pedidoId.toString()`, garantindo que todos os eventos do mesmo pedido caem na mesma partição e preservam ordem por agregado.

3.3.2 MySQL

Uma única instância MySQL 8.4 hospeda **quatro databases** isolados, um por microserviço — abordagem *database per service*. O script `init.sql` cria os quatro databases na primeira inicialização do container. O ADR 0007 detalha a decisão de manter os bancos isolados, mas na mesma instância MySQL.

O MySQL recebe tuning explícito no docker-compose:

- `innodb-buffer-pool-size=512M` (default é 128 MB)
- `innodb-redo-log-capacity=128M`
- `innodb-flush-log-at-trx-commit=2` (latência menor; tolerância de até 1 segundo perdido em crash do host)
- `innodb-flush-method=O_DIRECT` (evita double-buffering)
- `skip-name-resolve` (sem reverse DNS por conexão)

3.3.3 procpag

Gateway de pagamento externo fornecido pelos professores (imagem `docker.io/erickemprobr/procpag`). Simula um serviço eventualmente disponível, alternando respostas de aprovação com erros e timeouts. Toda a estratégia de resiliência (capítulo 5) gira em torno dessa imprevisibilidade controlada.

4 FLUXO PRINCIPAL

O fluxo feliz (sem falhas) do sistema percorre, em ordem, os passos descritos a seguir. A Figura 4 apresenta a mesma narrativa em forma gráfica como diagrama de sequência. A fonte Mermaid está em `docs/diagramas/sequencia-happy-path.md`.

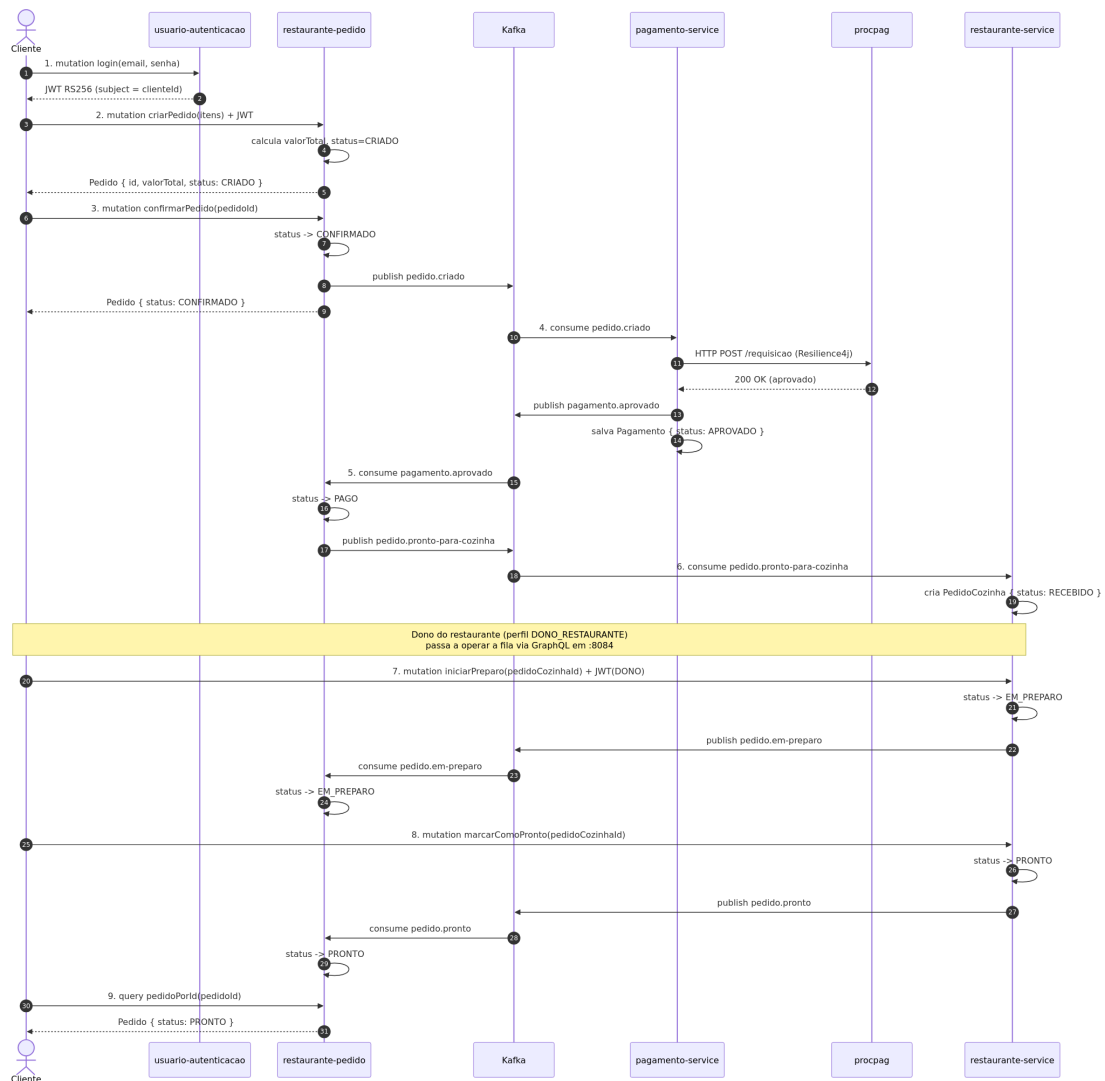


Figura 4 — Sequência do fluxo feliz (cadastro → entrega pela cozinha).

1. O cliente envia mutation login ao *usuario-autenticacao*, que valida credenciais e devolve um JWT RS256 contendo nas claims o subject (UUID do cliente) e a claim groups com seu perfil.

2. Com o token no header Authorization: Bearer <jwt>, o cliente envia mutation criarPedido ao *restaurante-pedido*. O serviço calcula o valor total a partir dos itens, persiste o pedido no status **CRIADO** e devolve o agregado completo. O *clientId* é sempre extraído do JWT, nunca recebido do cliente (req. 5.2).

3. O cliente envia mutation confirmarPedido. O status passa para **CONFIRMADO** e o evento *pedido.criado* é publicado no Kafka. O cliente recebe a resposta em milissegundos.

4. O *pagamento-service* consome *pedido.criado*, persiste um registro de pagamento e chama o *procpag* via HTTP. A chamada é envolvida por Resilience4j (CB + Retry + Timeout). No caminho feliz, o *procpag* responde 200 OK.

5. O *pagamento-service* persiste o pagamento como **APROVADO** e publica `pagamento.aprovado`.

6. O *restaurante-pedido* consome o evento, transita o pedido para **PAGO** e publica `pedido.pronto-para-cozinha` com os itens (sem preço — irrelevante para a cozinha).

7. O *restaurante-service* consome `pedido.pronto-para-cozinha` e cria o agregado `PedidoCozinha` no estado **RECEBIDO**.

8. O dono do restaurante consulta a fila via query `filaCozinha` em `:8084/graphql` e aciona mutation `iniciarPreparo`. O agregado passa a **EM_PREPARO**; o evento `pedido.em-preparo` é publicado.

9. O *restaurante-pedido* consome o evento e atualiza o pedido principal para **EM_PREPARO**. Quando o preparo termina, o dono aciona `marcarComoPronto`, o evento `pedido.pronto` propaga e o ciclo se fecha com o pedido em **PRONTO**.

4.1 Máquina de estados do agregado Pedido

Todas as transições descritas acima são reguladas pela máquina de estados implementada no agregado `Pedido` do módulo *restaurante-pedido*. A Figura 5 mostra os estados válidos e as transições permitidas; estados terminais (**PRONTO** e **CANCELADO**) não admitem saída. Todas as transições disparadas por eventos Kafka são **idempotentes** — receber o mesmo evento duas vezes não causa efeito colateral.

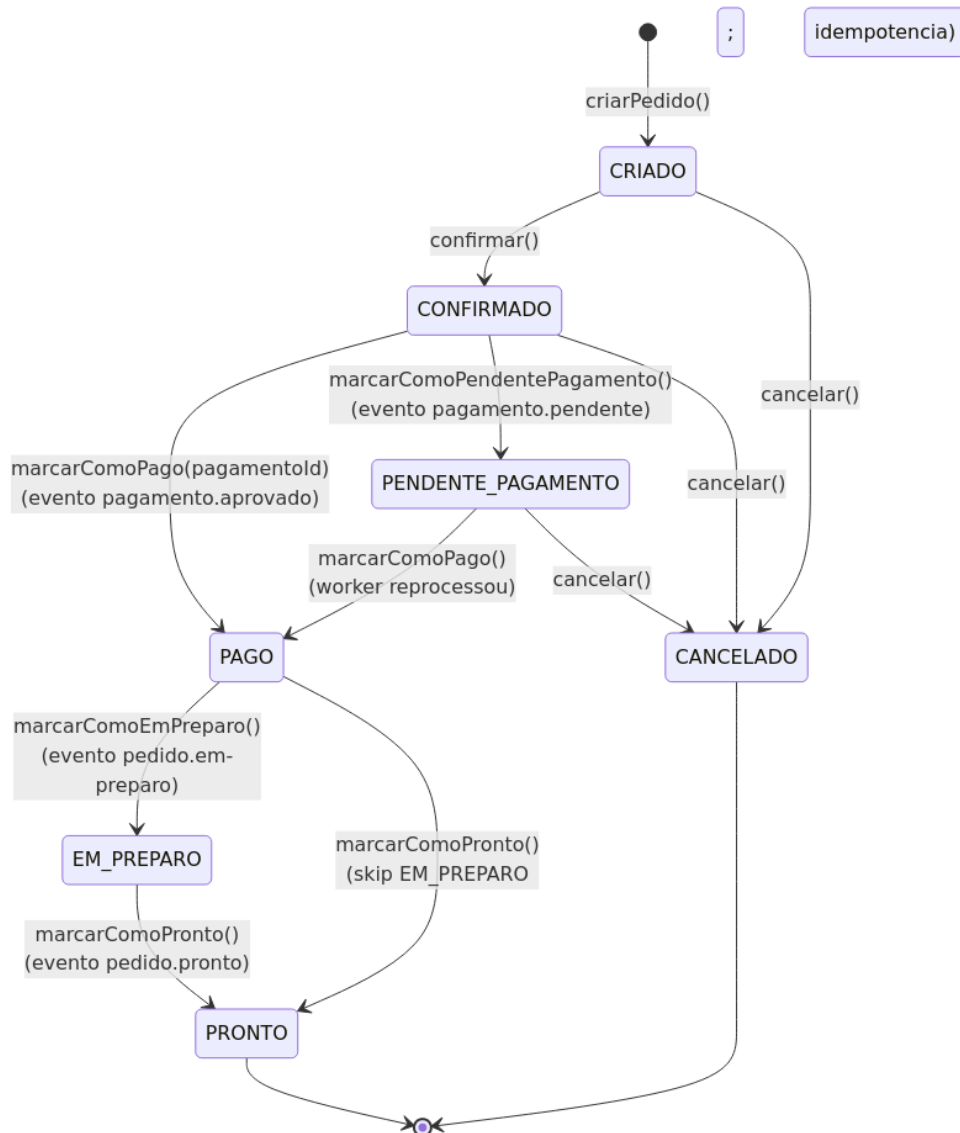


Figura 5 — Máquina de estados do agregado Pedido.

5 GUIA DE INTEGRAÇÃO

Esta seção descreve o contrato de API exposto pelo sistema: como uma aplicação cliente faz para autenticar, criar pedidos e consultar status. A referência completa (com payloads detalhados, exemplos e erros) está em `docs/api-integracao.md` no repositório; este capítulo apresenta o panorama essencial.

5.1 Visão geral do contrato

Todos os quatro microsserviços expõem uma **API GraphQL** no endpoint `POST /graphql`, autenticada via JWT no header `Authorization: Bearer <token>`. As únicas exceções são as mutations `cadastrarUsuario` e `login` do `usuario-autenticacao`, que não exigem token. A estrutura de cada chamada segue o padrão GraphQL:


```
{"query":"<query ou mutation>","variables":{...}}
```

Resposta segue o mesmo padrão, com campos **data** (sucesso) e **errors** (presente apenas em falhas).

5.2 Endpoints disponíveis

A Tabela 1 resume os endpoints e seu nível de autenticação. A Tabela 2 lista as 12 operações disponíveis no sistema.

Serviço	Endpoint	Autenticação
usuario-autenticacao	POST :8081/graphql	Não (cadastrarUsuario, login); JWT (me)
restaurante-pedido	POST :8082/graphql	JWT (USUARIO ou DONO_RESTAURANTE)
pagamento-service	POST :8083/graphql	JWT
restaurante-service	POST :8084/graphql	JWT com perfil DONO_RESTAURANTE

Operação	Serviço	Quando usar
mutation cadastrarUsuario	auth	Criar conta nova
mutation login	auth	Obter JWT (validade 1h)
query me	auth	Ver dados do usuário corrente
mutation criarPedido	pedido	Criar pedido no status CRIADO
mutation confirmarPedido	pedido	Confirmar e disparar processamento
query pedidoPorId	pedido	Consultar (com ownership check)
query meusPedidos	pedido	Listar pedidos do cliente do JWT
query pagamentoPorPedido	pagamento	Status do pagamento
query pagamentosPendentes	pagamento	Lista da fila de reproprocessamento
query filaCozinha	cozinha	Listar fila por status
mutation iniciarPreparo	cozinha	RECEBIDO → EM_PREPARO
mutation marcarComoPronto	cozinha	EM_PREPARO → PRONTO

5.3 Fluxo recomendado de integração

Para uma aplicação cliente integrar do zero, a sequência padrão é:

- **1. cadastrarUsuario** (apenas se a conta ainda não existe; pode ser pulado se já houver registro).
- **2. login** — guarda o token retornado.
- **3. criarPedido** com o JWT no header. Devolve o pedido em CRIADO, com valor total já calculado pelo servidor.
- **4. confirmarPedido**. Publica o evento Kafka `pedido.criado`, disparando o fluxo de pagamento em background.

- 5. Polling com `pedidoPorId` a cada poucos segundos. Em ~5 s, o status transita para PAGO (caminho feliz) ou PENDENTE_PAGAMENTO (gateway externo fora).
- 6. Para clientes do tipo dono do restaurante, `filaCozinha + iniciarPreparo + marcarComoPronto` avançam o pedido até PRONTO.

5.4 Classificação dos erros

Erros não usam códigos HTTP convencionais por padrão — a resposta vem em 200 OK com array `errors`. A classificação fica em `errors[].extensions.classification`:

classification	Significado	HTTP equivalente
BAD_REQUEST	Validação, transição de estado proibida	400
UNAUTHORIZED	JWT ausente, expirado ou inválido	401
FORBIDDEN	Perfil insuficiente para a operação	403
NOT_FOUND	Recurso inexistente	404
INTERNAL_ERROR	Erro inesperado no servidor	500

O *restaurante-pedido* tem um adapter adicional (`GraphQLHttpStatusConfig`) que traduz a `classification` para o status HTTP de fato, facilitando integração com clientes REST tradicionais.

5.5 Como descobrir o schema em runtime

GraphQL é introspectivo. Há três formas de descobrir o schema em runtime:

- **GraphiQL embutido** — console interativo em `:8081/graphiql`, `:8082/graphiql`, `:8083/graphiql` e `:8084/graphiql`. Clique em "Docs" no canto superior direito para navegar pelo schema.
- **Introspection query** — POST padrão com `{ __schema { types { name kind } } }`.
- **OpenAPI/Swagger** (apenas no *restaurante-pedido*) — contrato HTTP documentado em `http://localhost:8082/swagger-ui.html`, útil para clientes REST tradicionais.

6 RESILIÊNCIA E TOLERÂNCIA A FALHAS

A resiliência do sistema concentra-se na integração com o gateway externo *procpag*, ponto único de instabilidade previsível. Toda a estratégia foi implementada com **Resilience4j 2.3.0** via anotações no método de integração HTTP, e complementada por um worker agendado para reprocessamento automático. A Figura 6 ilustra o fluxo completo de falha e recuperação.

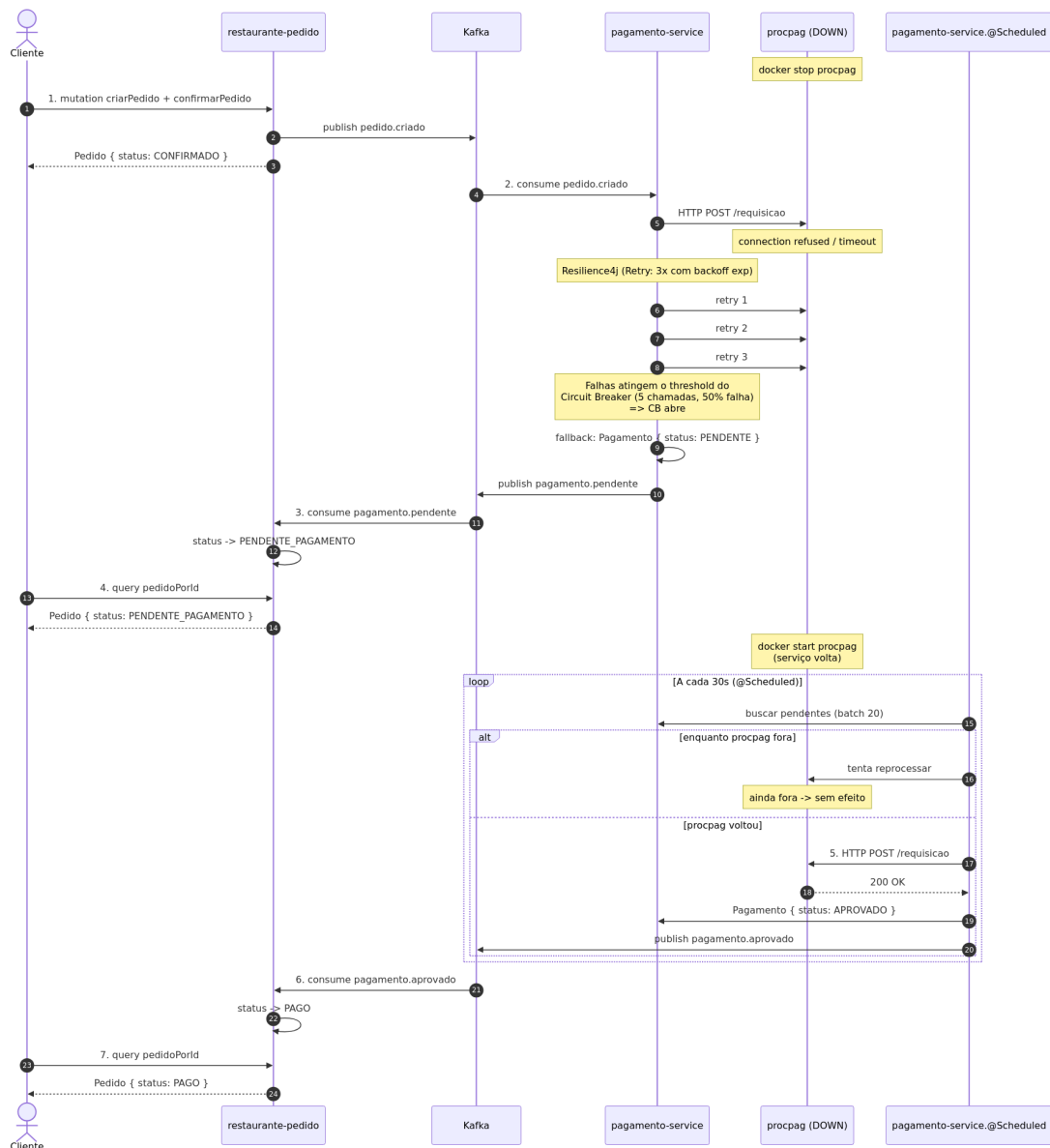


Figura 6 — Sequência de falha do gateway + reprocessamento automático.

6.1 Padrões aplicados

Os quatro padrões clássicos de resiliência foram aplicados em camadas:

- **Retry** — três tentativas com backoff exponencial (2s, 4s, 8s).
- **Timeout** — cinco segundos por chamada; chamadas mais lentas são interrompidas.
- **Circuit Breaker** — abre após 50% de falhas em uma janela deslizante de 5 chamadas. Quando aberto, novas chamadas falham imediatamente sem tocar o gateway. Após 30 segundos, transita para HALF_OPEN e testa 2 chamadas; se passarem, fecha.
- **Fallback** — quando todas as estratégias acima falham, o pagamento é marcado como **PENDENTE**, e o evento `pagamento.pendente` é publicado no Kafka. O pedido nunca falha do ponto de vista do cliente.

6.2 Worker de reprocessamento

Vive exclusivamente no módulo *pagamento-service*. É um `@Scheduled` que executa a cada 30 segundos (configurável em `pagamento.reprocess.fixed-delay-ms`), busca um lote de até 20 pagamentos no estado PENDENTE e retenta cada um deles contra o procpag. Quando o gateway voltar e a chamada for bem-sucedida, o pagamento é marcado APROVADO e `pagamento.aprovado` é publicado — fechando o ciclo de recuperação.

Por que o worker está no pagamento-service, e não no restaurante-pedido? O worker opera sobre dados do próprio bounded context (registros de pagamento). Reprocessar significa tentar de novo a chamada ao gateway, integração que vive no pagamento-service e em mais lugar nenhum. Manter o worker fora do *restaurante-pedido* preserva a separação de responsabilidades: o *restaurante-pedido* apenas reage a eventos.

6.3 Garantias e limitações

O sistema garante **at-least-once** na entrega de eventos Kafka (o consumer pode receber o mesmo evento mais de uma vez em cenários de reprocessamento). As transições de estado nas entidades de domínio são **idempotentes**: marcar um pedido já PAGO como PAGO novamente não causa efeito. Isso é exercitado nos testes unitários do agregado *Pedido*.

Não há, no escopo atual, mecanismo de **outbox** ou transações distribuídas entre o commit do banco local e a publicação no Kafka. Em cenário de crash entre as duas operações, há risco teórico de inconsistência (o pedido vira PAGO no banco mas o evento não é publicado). Para o escopo do projeto, esse risco é considerado aceitável.

7 SEGURANÇA

7.1 Autenticação e autorização

A autenticação é baseada em **JWT RS256** emitido pelo *usuario-autenticacao*. A chave privada RSA vive apenas neste serviço; a chave pública é distribuída no classpath dos demais (`src/main/resources/keys/publicKey.pem`), permitindo validação local sem necessidade de chamar o *usuario-autenticacao* a cada requisição. O ADR 0008 detalha a escolha pela assinatura assimétrica.

A autorização usa **perfis** codificados na `claim groups` do JWT: USUARIO (cliente final) e DONO_RESTAURANTE (administrador da fila da cozinha). Cada resolver GraphQL é protegido por `@PreAuthorize`, com regras específicas para cada operação.

7.2 Ownership check em pedidos

Na query `pedidoPorId`, o serviço extrai o `clienteId` do JWT e compara com o do pedido buscado. Pedidos de outros clientes retornam `null` — a mesma resposta de 'não encontrado', para não vazar a existência do recurso. Esta verificação foi adicionada como melhoria de segurança identificada em auditoria de conformidade.

8 DECISÕES ARQUITETURAIS

Cada decisão técnica significativa do projeto está registrada como ADR (Architecture Decision Record) na pasta `docs/adr/` do repositório, no formato proposto por Michael Nygard (Contexto + Decisão + Consequências). O índice completo está em `docs/adr/README.md`.

São 13 ADRs vigentes:

ADR	Decisão	Status
0001	Adoção de microsserviços em vez de monolito	Accepted
0002	Arquitetura hexagonal em cada microsserviço	Accepted
0003	GraphQL no ponto de entrada das aplicações	Accepted
0004	gRPC para chamadas síncronas entre serviços	Accepted
0005	Apache Kafka para comunicação assíncrona	Accepted
0006	Kafka em modo KRaft (sem Zookeeper)	Accepted
0007	Database separado por serviço (mesmo MySQL)	Accepted
0008	JWT com assinatura assimétrica RS256	Accepted
0009	Resilience4j para resiliência na integração	Accepted
0010	Bounded context separado para a cozinha	Accepted
0011	Docker Compose como orquestrador	Accepted
0012	AppCDS + Layered JARs no Dockerfile	Accepted
0013	Resource limits explícitos no compose	Accepted

Os ADRs descrevem não apenas a decisão tomada mas também as **alternativas consideradas e descartadas**, preservando o raciocínio que conduziu à escolha. Quando uma decisão for revisitada no futuro, o ADR existente receberá status *Superseded by ADR-XXXX* e um novo ADR será criado.

9 ESTRATÉGIA DE TESTES

A suíte de testes do projeto totaliza **285 testes automatizados** distribuídos pelos quatro módulos, executando em aproximadamente três minutos. Toda a suíte roda sem dependências externas — utiliza H2 em memória no lugar do MySQL, `@EmbeddedKafka` no lugar do broker real e mocks para integrações HTTP.

Módulo	Testes	Cobertura
usuario-autenticacao	63	Domain, use cases, JWT, gRPC, GraphQL, smoke
restaurante-pedido	126	Domain, 4 consumers, 2 publishers, GraphQL, smoke
pagamento-service	57	Domain, Resilience4j, worker, GraphQL, smoke
restaurante-service	39	Domain, 4 use cases, JPA, Kafka, GraphQL, smoke
TOTAL	285	

A pirâmide de testes adotada privilegia **testes unitários** rápidos (JUnit 5 + Mockito + AssertJ) para a maior parte da cobertura, complementados por **smoke tests** de contexto Spring (`@SpringBootTest`) e por **testes de integração** da camada GraphQL via MockMvc. O domínio puro permite que regras de negócio sejam testadas sem qualquer dependência de framework.

10 OPERAÇÃO E DEPLOY

Todo o sistema é orquestrado por Docker Compose. O comando único de subida é:

```
docker compose up -d --build
```

Esse comando constrói as imagens das quatro aplicações (quando necessário) e sobe os oito containers. O primeiro build é demorado porque cada Dockerfile executa um *training run* de AppCDS para gerar o archive de classes compartilhadas (ver ADR 0012). Execuções subsequentes aproveitam o cache de camadas e ficam em poucos segundos.

10.1 Limites de recursos

Cada serviço tem limites explícitos de memória e CPU configurados em `deploy.resources.limits`. O total combinado fica em aproximadamente 3,5 GB de RAM e 7,5 vCPUs, permitindo execução em máquinas convencionais sem comprometer o sistema operacional do host. O ADR 0013 detalha a calibração.

10.2 Observabilidade

Cada aplicação expõe Spring Boot Actuator nos endpoints `/actuator/health`, `/actuator/info` e `/actuator/metrics`. O serviço *pagamento-service* expõe adicionalmente `/actuator/circuitbreakers` (estado dos breakers Resilience4j), `/actuator/retries` e `/actuator/scheduledtasks` (confirma o worker ativo). Os tópicos Kafka podem ser inspecionados via Kafka UI em `http://localhost:8085`.

10.3 Performance

Três otimizações foram aplicadas para reduzir o consumo de recursos e acelerar o startup das aplicações Spring:

- **Kafka em modo KRaft** — eliminou o container Zookeeper, reduzindo em ~150 MB o consumo de RAM (ADR 0006).
- **AppCDS** — pré-resolve classes em um archive binário gerado durante o docker build, reduzindo o boot da JVM em aproximadamente 30 por cento (ADR 0012).
- **Layered JARs** — o jar é particionado em camadas semânticas, permitindo cache mais inteligente do Docker em rebuilds: mudança de código invalida apenas a camada de aplicação (~5 MB), não o jar completo (~50 MB) (ADR 0012).

11 CONSIDERAÇÕES FINAIS

A arquitetura desenhada cumpre integralmente os requisitos da Fase 3 e ainda implementa o módulo opcional *restaurante-service*, expandindo o fluxo do pedido até a entrega pela cozinha. Os principais ganhos da abordagem distribuída — isolamento de falhas, escalabilidade granular, evolução independente — são contrabalançados pelo aumento da complexidade operacional, que se manifesta em oito containers, quatro databases e um broker de mensageria coordenando tudo.

Decisões pontuais foram documentadas em ADRs para preservar o raciocínio que conduziu a cada escolha, permitindo que evoluções futuras tenham contexto completo. Diagramas em formato Mermaid acompanham esta documentação na pasta `docs/diagramas/`.

Como pendência identificada e não resolvida no escopo da Fase 3, registra-se a ausência de mecanismo de **outbox** para garantir atomicidade entre commit do banco local e publicação de eventos Kafka, e a não implementação de **correlation IDs** para rastreabilidade distribuída de requisições — itens naturais para uma próxima iteração.

O sistema está pronto para demonstração, com suíte verde de 285 testes e roteiro de apresentação detalhado em `docs/roteiro-video.md`.

REFERÊNCIAS

COCKBURN, Alistair. **Hexagonal Architecture**. 2005. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>. Acesso em: 26 May. 2026.

EVANS, Eric. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2003.

NYGARD, Michael. **Documenting Architecture Decisions**. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 26 May. 2026.

PIVOTAL SOFTWARE. **Spring Boot Reference Documentation**. 2025. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. Acesso em: 26 May. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Kafka Documentation**. 2025. Disponível em: <https://kafka.apache.org/documentation/>. Acesso em: 26 May. 2026.

RESILIENCE4J. **Resilience4j User Guide**. 2025. Disponível em: <https://resilience4j.readme.io/>. Acesso em: 26 May. 2026.

FOWLER, Martin. **Microservices**. 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 26 May. 2026.

OPENJDK. **JEP 310: Application Class-Data Sharing**. 2018. Disponível em: <https://openjdk.org/jeps/310>. Acesso em: 26 May. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro, 2011.

FIAP. **Tech Challenge — Fase 3**. Material institucional do programa de Pós-Graduação PosTech. São Paulo, 2025.